Deep Learning (AI-3102): Course Overview

These notes cover weeks 1 to 5 of the Deep Learning course. The course moves from single neurons to sequence models, with a focus on understanding why each technique exists, not only how to use it.

Assessment breakdown

- Quizzes: 15%
- Programming assignments: 25%
- Midterm exam: 25%
- Final exam: 35%

Course policies

Late assignments lose 10% of their marks for each day they are late, and are not accepted more than 3 days after the deadline. All assignments must be written in PyTorch. Group work is allowed only for the final assignment, in groups of at most two students.

Instructor note: Quiz 2 covers backpropagation and the LR halving rule (page 3). Bring a calculator.

Week 1: Neural Network Basics

The perceptron

A perceptron computes a weighted sum of its inputs, adds a bias, and passes the result through an activation function: output = f(w_1x_1 + w_2x_2 + ... + b). A single perceptron can only separate data with a straight line, which is why it cannot solve the XOR problem.

Activation functions

- **Sigmoid** squashes values to the range 0 to 1. It is useful for binary output layers but suffers from vanishing gradients in deep networks.
- **Tanh** squashes values to the range -1 to 1 and is zero-centred, which often trains faster than sigmoid.
- **ReLU** outputs max(0, x). It is cheap to compute and is the default choice for hidden layers. Its weakness is 'dying ReLU', where a neuron outputs zero for every input and stops learning.

Why non-linearity matters

Stacking linear layers without activation functions collapses into a single linear layer, no matter how many layers you add. Non-linear activations are what allow deep networks to learn complex patterns.

Week 2: Training Neural Networks

Loss functions

Mean squared error (MSE) is used for regression. Cross-entropy loss is used for classification, because it heavily penalises confident wrong predictions.

Gradient descent and backpropagation

Gradient descent updates each weight in the direction that reduces the loss: w = w - learning_rate * gradient. Backpropagation computes these gradients efficiently by applying the chain rule backwards from the loss to every layer. Mini-batch gradient descent, which uses small batches such as 32 or 64 samples, is the standard compromise between speed and stability.

Choosing the learning rate

A learning rate that is too high makes the loss jump around or explode. One that is too low makes training very slow. Adam is the default optimizer in this course because it adapts the learning rate for each parameter.

The LR halving rule (used in all assignments): if the validation loss increases for 3 consecutive epochs, halve the learning rate. If it has been halved 4 times, stop training.

Week 3: Regularization

Overfitting

A model overfits when it memorises the training data instead of learning general patterns. The sign is a training loss that keeps falling while the validation loss starts rising.

Techniques to reduce overfitting

- **Dropout:** randomly switches off a fraction of neurons during training, usually between 0.2 and 0.5. Dropout is turned off during inference.
- **L2 regularization (weight decay):** adds a penalty on large weights to the loss, keeping the model simpler.
- **Early stopping:** stop training when validation loss stops improving and keep the best checkpoint.
- **Data augmentation:** create new training samples by flipping, cropping or rotating images.

Assignment 2 requires dropout of exactly 0.3 in every hidden layer, so that results are comparable across students.

Week 4-5: Sequence Models and Attention

Recurrent neural networks

An RNN processes a sequence one step at a time, passing a hidden state forward. Training long sequences is difficult because gradients shrink as they flow back through many steps, the vanishing gradient problem.

LSTM

The LSTM fixes vanishing gradients with a cell state controlled by three gates: the forget gate decides what to erase, the input gate decides what new information to store, and the output gate decides what to expose as the hidden state.

Attention in sequence-to-sequence models

In a basic encoder-decoder, the whole input sentence is squeezed into one fixed vector, which loses information for long sentences. Attention (Bahdanau, 2014) lets the decoder look back at every encoder hidden state and take a weighted average of them at each output step. In this course, attention is studied as an add-on to RNNs; the NLP course covers self-attention and transformers.